

Mobile Integration Guide: API Encryption & Authentication

This document describes two independent mechanisms the mobile client must implement to talk to the .NET API on protected routes:

- 1. **Payload encryption** (AES-256-CBC) for endpoints that opt in
- 2. **Bearer token authentication** using an opaque session token

These mechanisms are orthogonal — a call may require one, both, or neither, depending on the endpoint. This is a reference document; it describes the wire contract, not a specific mobile framework's implementation.

Section 1: Payload Encryption

1.1 Cipher Specification

Parameter	Value
Algorithm	AES-256-CBC
Padding	PKCS7
Key	32-byte (256-bit) UTF-8 string, used directly as raw key bytes
Key derivation	None — no KDF, no hashing, no salting. The key string's bytes <i>are</i> the AES key.
IV	16 random bytes, generated fresh for every encryption call

The 32-byte key value ("the shared AES key") will be supplied out of band by the backend team. It must be stored securely on-device (see Section 2.1 for the equivalent guidance that applies here too) and never hardcoded in source control in production builds.

IV reuse is not acceptable. A new random IV must be generated per encryption operation, including when encrypting multiple requests with identical bodies.

1.2 Wire Format

The IV and ciphertext are each base64-encoded independently, then joined with a single colon:

```
<ivBase64>:<cipherTextBase64>
```

This string is then wrapped in a JSON envelope for both requests and responses:

```
json
```

```
{ "payload": "<ivBase64>:<cipherTextBase64>" }
```

The plaintext that gets encrypted is the original JSON request/response body, UTF-8 encoded before encryption. In other words, the flow is:

```
originalBodyJson (UTF-8 bytes)
  → AES-256-CBC encrypt (random IV, shared key)
  → base64(iv) + ":" + base64(ciphertext)
  → { "payload": "<that string>" }
```

Decryption on the client (for encrypted responses) reverses this: split on the colon, base64-decode both parts, decrypt with the shared key and the given IV, UTF-8-decode the result, parse as JSON.

1.3 When Encryption Applies

Encryption is **opt-in per request**, controlled by a header:

```
X-Encrypted: true
```

- If the client sends this header, both the request body **and** the expected response body are the encrypted envelope described above.
- If the client does **not** send this header, the request and response bodies are plain, unencrypted JSON.
- This allows certain endpoints (health checks, external/third-party integrations, etc.) to opt out of encryption entirely.

Client responsibility: the mobile app must explicitly set `X-Encrypted: true` on every call it wants encrypted, and must be prepared to send and parse plain JSON on any call where it omits the header. There is no global mode — this is decided per request.

1.4 Multipart / File Upload Exemption

Multipart/form-data requests (file uploads, etc.) are **exempt** from encryption:

- Do **not** encrypt multipart bodies.
- Do **not** set `X-Encrypted: true` on multipart requests.
- These requests pass through as standard `multipart/form-data`.

1.5 Decryption Failure Behavior

If the server cannot decrypt an incoming request — malformed envelope, wrong key, corrupted/truncated payload — it responds:

Client handling: treat a 400 on an encrypted call as a signal to check the client-side encryption implementation (key correctness, IV format, envelope structure), not as a transient error. Do not blindly retry the same request; retrying an implementation bug will just produce the same 400.

Section 2: Authentication

2.1 Session Token

After a successful login call, the response body contains an opaque session token:

```
json
{
  "Data": {
    "tokens": "..."/>

```

Key characteristics:

- This is **not a JWT**. It is an opaque string that the server looks up against its own session store — the client should not attempt to decode, parse, or infer expiry from it.
- The login call itself is unencrypted (plain JSON) unless the backend team specifies otherwise for that specific endpoint — confirm with backend team if this changes.

2.2 Client Responsibilities

1. **Store the token securely** after login — platform secure storage only (e.g., Keychain, EncryptedSharedPreferences, or equivalent secure enclave/keystore mechanism for the platform in use). Do not persist it in plain shared preferences, plist files, local storage, or logs.
2. **Send it on every subsequent protected API call** as a standard bearer token:

```
Authorization: Bearer <token>
```

3. **Treat `401 Unauthorized` as "session invalid or expired."** Standard flow:
 - Clear the locally stored token.
 - Redirect the user to the login screen.
 - Do not silently retry the request or loop on 401s.

Token refresh flows and token lifetime/expiry policy are not specified here — confirm with backend team.

2.3 401 Response Shape

When a call fails auth, the server returns a JSON body (itself unencrypted, since auth is checked independently of the encryption layer) shaped like:

```
json
{
  "Message": "...",
  "IsSuccess": false,
  "ResponseCode": 401
}
```

The client may surface `Message` directly to the user where appropriate, rather than showing a generic error string, since the server provides a human-readable reason.

Section 3: Combining Both Mechanisms

Encryption and authentication are independent layers and are both applied, where required, on the same request:

- `Authorization: Bearer <token>` — present on every call to a protected route.
- `X-Encrypted: true` + encrypted `payload` envelope — present only if that specific endpoint requires encryption.

A protected, encrypted endpoint therefore expects **both** headers and an encrypted body simultaneously. Neither mechanism substitutes for the other: a valid bearer token does not exempt a request from encryption, and an encrypted payload does not exempt a request from needing a valid token.

3.1 End-to-End Example

Step 1 — Login (unencrypted, no token yet)

Request:

```
POST /api/auth/login
Content-Type: application/json

{
  "username": "jdoe",
  "password": "●●●●●●●●"
}
```

Response:

```
json
{
  "IsSuccess": true,
  "ResponseCode": 200,
  "Message": "Login successful",
  "Data": {
    "tokens": "a1b2c3d4-opaque-session-token"
  }
}
```

The client stores `a1b2c3d4-opaque-session-token` in secure storage.

Step 2 — Subsequent protected + encrypted call

The client encrypts its JSON body using the shared AES key and a freshly generated IV, builds the `payload` envelope, and sends:

Request:

```
POST /api/orders
Authorization: Bearer a1b2c3d4-opaque-session-token
X-Encrypted: true
Content-Type: application/json

{
  "payload": "cmFuZG9tSVZieXRlcw==:c29tZUNpcGhlcnRleHRCeXRlcw=="
}
```

The server validates the bearer token, decrypts `payload` with the shared key, and processes the decrypted JSON. It then encrypts its own JSON response body the same way and returns:

Response:

```
json
{
  "payload": "YW5vdGhlcklwYn10ZXM=:cmVzcG9uc2VDaXBoZXJ0ZXh0Qn10ZXM="
}
```

The client decrypts `payload` (split on `:`, base64-decode both parts, AES-256-CBC decrypt with the shared key and the given IV, UTF-8-decode, parse JSON) to get the actual

response data.

If the token had expired instead: the server would return `401` with the `Message` / `IsSuccess: false` / `ResponseCode: 401` shape from Section 2.3, and the encrypted body would not be relevant — the client should clear the stored token and redirect to login as described in Section 2.2.

Open Items to Confirm with Backend Team

- Token expiry/lifetime and whether any refresh mechanism exists.
- Retry policy (if any) for transient (non-400/401) failures.
- Rate limiting behavior on auth or encrypted endpoints.
- Whether the login endpoint or other specific endpoints ever require encryption.